

Strongly typed view

В предишните уроци видяхме как тече прехвърляне на малки пакети от данни от контролера към изгледа чрез ViewBag и ViewData и техните слаби места – на първо място, че при тях не се включва IntelliSense, който да разпознае автоматично, че дадените свойства принадлежат на сложния обект и лесно се допускат правописни грешки като изпускане на буква, а тя няма да бъде хваната от компилатора, а директно в runtime при зареждане на страницата – или няма да се покаже, или цялата страница ще крашне. Затова, за мънички пакети като заглавие на страница ViewBag и ViewData са подходящи, но дотам. Ако се прехвърлят важни данни, както в случая са данните на даден служител, много по-добре е да сме сигурни какво точно пишем и включим автоматично разпознаване чрез IntelliSense. Това става, като декларираме откъде точно вземаме данните на сложния обект.

За целта на първи ред използваме с малка буква @model + директория

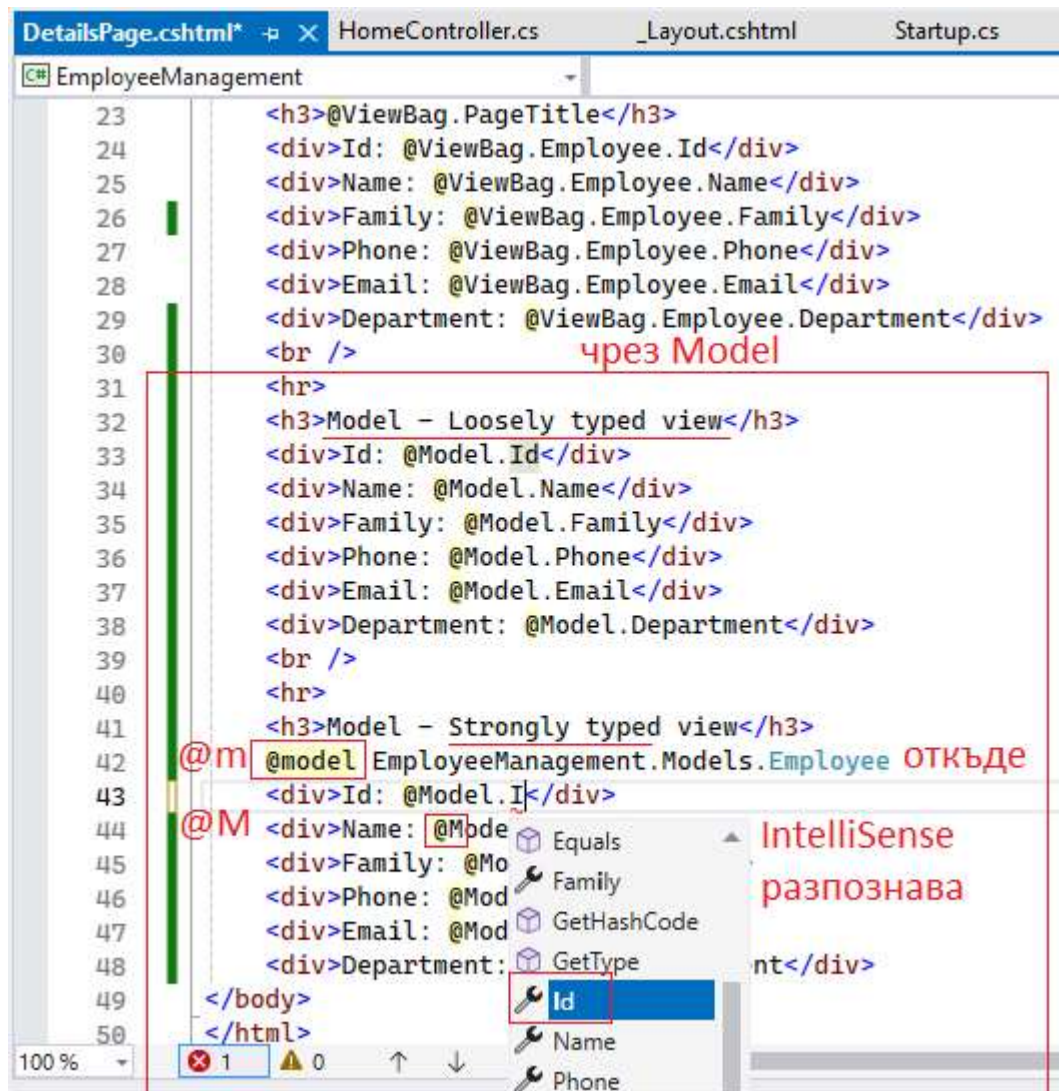
@model EmployeeManagement.Models.Employee

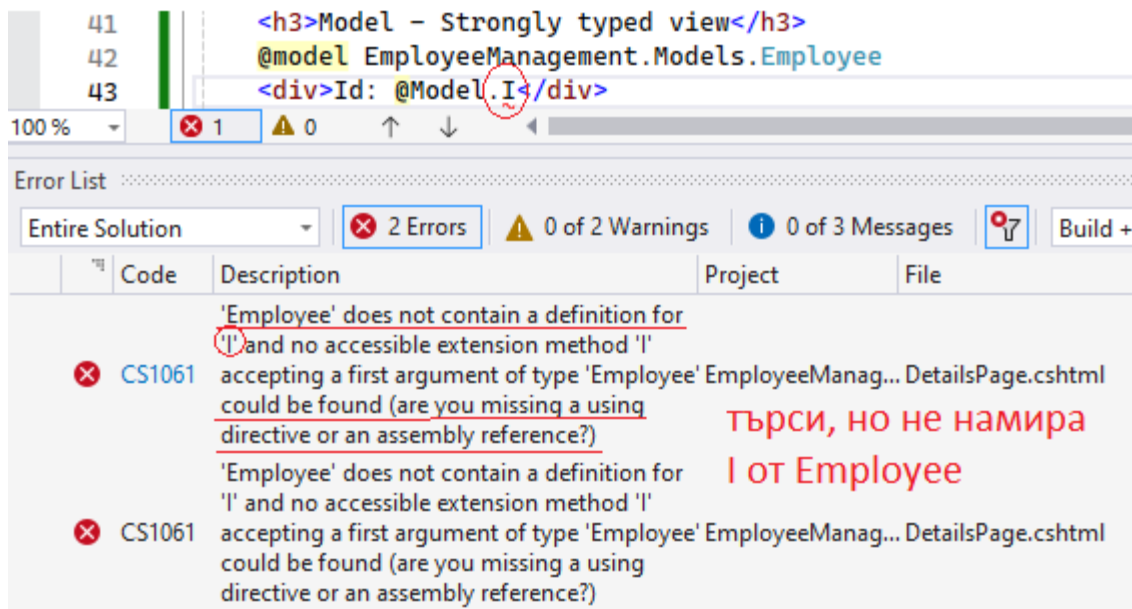
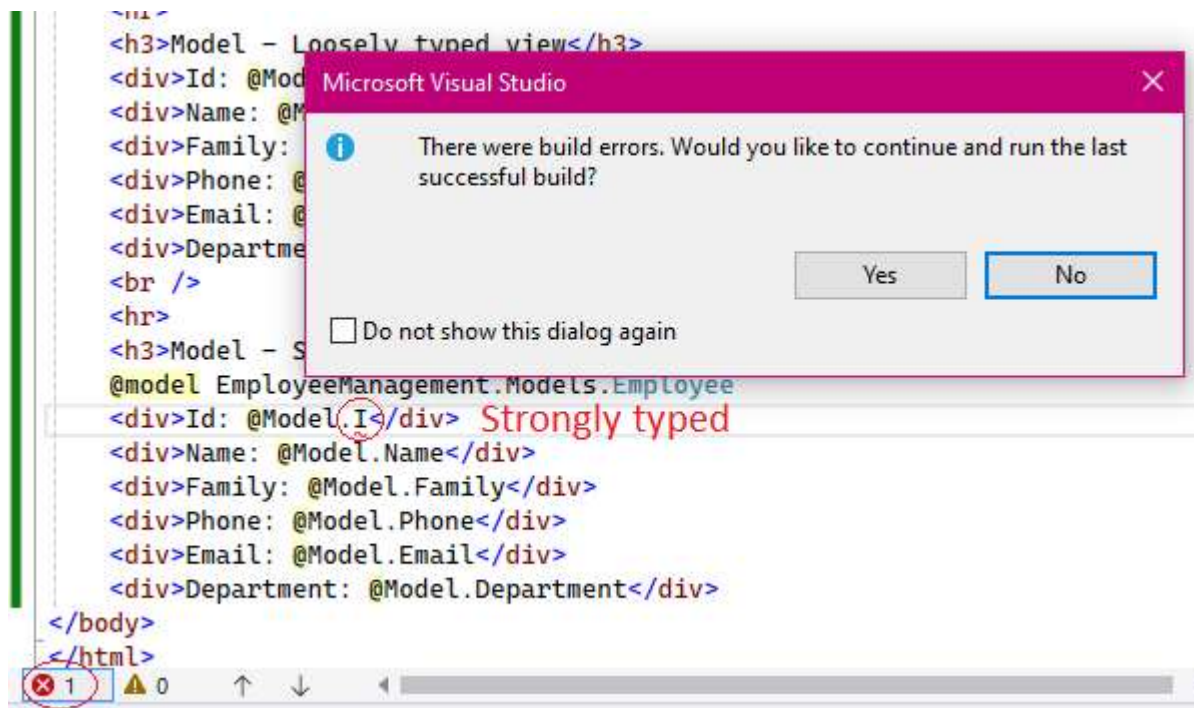
т.е. указваме откъде да се включи библиотеката, класът, в който се намира този сложен тип данни, нещо като namespace

Strongly Type = ред 1: @model EmployeeManagement.Models.Employee – дава големи възможности за писане на код без грешки с избор на всички допустими опции и корекции на грешки за по-добра производителност.

Strongly Typed View provides compile-time type checking + IntelliSense.

В случая, с демонстративна цел, първия път просто извикваме работещ код с @Model, а втория път вече чрез включване на библиотеката локално вътре в самия код – принципно не се прави така, а се слага най-отгоре, на първия ред на страницата, за да може да се извика където и да е под него. Ние, обаче, искаме да тестваме веднъж с изключен и веднъж с включен на IntelliSense, затова ще го изпишем точно преди да ни потрябва:





Когато коригираме на Id:

```
<h3>Model - Strongly typed view</h3>
@model EmployeeManagement.Models.Employee
<div>Id: @Model.Id</div>
<div>Name: @Model
<div>Family: @Mod int EmployeeManagement.Models.EmployeeId { get; set; }
<div>Phone: @Model.Phone</div>
<div>Email: @Model.Email</div>
<div>Department: @Model.Department</div>
</body>
No issues found
```

Вече го открива и предоставя информация за него, докато, ако дадем курсора върху някой от предишните Id, изписва, че има динамично предаване на данни, но не разпознава откъде:

```
<hr>
<h3>@ViewBag.PageTitle : ViewBag</h3>
<div>Id: @ViewBag.Employee.Id</div> не го разпознава
<div>Name: @ViewBag.Employee.
<div>Family: @ViewBag.Employee
<div>Phone: @ViewBag.Employee
<div>Email: @ViewBag.Employee
<div>Department: @ViewBag.Employee.Department</div>
```

dynamic
Represents an object whose operations will be resolved at runtime.

Employee Details

Id: 1

Name: Мария

Family: Георгиева

Phone: 0123456789

Email: Maria@omg.bg

Department: Biology

Model - Loosely typed view

Id: 1

Name: Мария

Family: Георгиева

Phone: 0123456789

Email: Maria@omg.bg

Department: Biology

Model - Strongly typed view

Id: 1

Name: Мария

Family: Георгиева

Phone: 0123456789

Email: Maria@omg.bg

Department: Biology

To create a Strongly Typed View

- Specify model type in the view using **@model** directive

```
@model EmployeeManagement.Models.Employee
```

- To access the model object properties we use **@Model**

```
<div>  
    Name : @Model.Name  
</div>  
<div>  
    Email : @Model.Email  
</div>
```

малка буква 'm'

+

голяма буква 'M'

- Strongly Typed View provides **compile-time type checking** and **intellisense**